

Unity Netcode for GameObjects

Joe Ampfer, ASE 485 Capstone

What I learned with AI while turning Spell Arena into a multiplayer game.

What Is NGO?

Unity's official multiplayer package, also called **Netcode for GameObjects**.

It sits on top of a **transport layer** (the thing that actually moves bytes between machines) and gives you the high-level Unity-flavored building blocks:

- Spawning and despawning networked objects
- RPCs (covered in Topic 1)
- NetworkVariables for synced values
- Scene transitions for everyone at once
- Ownership of objects

This slide deck is the parts of NGO I actually used.

Spawning Networked Objects

When you **spawn** a GameObject through NGO, it doesn't just appear on your machine.

- A **duplicated copy** of the GameObject shows up on every connected client
- Any script on that GameObject runs **separately on each machine**
- So if a script does something visual, it happens on everyone's screen for free
- If a script does something authoritative (damage, scoring), you need to gate it so only the server runs it

Spawning is what makes "the world looks the same on everyone's screen" actually true.

Despawning

Despawning is the opposite: the object goes away on every client.

- Despawning fires a **callback** you can react to (`OnNetworkDespawn`)
- Great for "this thing is ending, do something cosmetic"
- Example: when a spell projectile despawns on impact, every client plays the **explosion VFX** in their copy of `OnNetworkDespawn`

The server destroys the object. The clients just react.

Same Code, Different Machines

Here is the thing that confused me the most at first.

In a host-client setup, the **server and the clients are running the exact same code**. Same scripts, same prefabs, same `Update()`.

So if a script says "*on key press, do X*", that code is alive on:

- The host's machine (which is also the server)
- Every connected client
- Every networked copy of the object

Without guards, all of them try to do X. NGO gives you two checks to fix this: `IsServer` and `IsOwner`.

IsServer — Gating Server-Only Logic

IsServer is true on the machine acting as the server.

Pattern I use everywhere:

```
if (!IsServer) return;
```

Anything below that line **only runs on the server**. I use this for:

- Applying damage to a player
- Deciding who wins
- Spawning networked objects (only the server is allowed to spawn anyway)

This is how the server stays the **source of truth** even though the client has the same code.

IsOwner — Gating Per-Player Logic

Every player has a character in the scene. Each character runs the **same script**.

Without an ownership check, here's what happens when I press F:

- **My character** sees the key press and casts a spell
- **The other player's character** also sees the key press and casts a spell

Two characters in my scene, both reacting to *my* keyboard. Wrong.

`if (!IsOwner) return;` fixes it. I "own" my character because NGO assigned it to me when I joined. So my character listens to my input, and the other one ignores it.

NetworkVariables

`NetworkVariable<T>` is a value that **automatically syncs** from the server out to all clients.

- Server changes the value
- Every client gets notified that it changed
- You can attach **as many listeners as you want** to react to that change
- All of them are decoupled — the listener doesn't know about the others

Health and mana all live in NetworkVariables in my game.

The Observer Pattern Is Everywhere

Once I noticed it, I couldn't stop seeing it. NGO is the **observer pattern over and over**:

- `NetworkVariable.OnValueChanged` — value changed
- `OnNetworkSpawn` / `OnNetworkDespawn` — object appeared / disappeared
- `NetworkManager.Singleton.OnClientConnectedCallback` — someone joined
- `OnClientDisconnectedCallback` — someone dropped
- `OnLoadEventCompleted` — the scene finished loading on everyone
- Steamworks lobby events (next section) — same idea

Same shape every time: subscribe a function, it gets called when a thing happens. Once you see it, the API stops feeling huge.

Steamworks: Free Transport

For my game I used **Facepunch.Steamworks** as the transport layer.

- Steam already runs a peer-to-peer connection system between players who own a Steam game
- I get to use that for **free**
- I don't pay for Unity Relay
- I don't host my own dedicated servers
- I don't ask players to port-forward

Steam moves the bytes. NGO does the game logic on top.

Steamworks: Lobbies

Steam also gives me **lobbies** out of the box.

- A lobby is a place players can gather **before anyone is hosting a match**
- I get matchmaking, invites, and shared lobby data (selected character, ready state, etc.) for free
- Lobbies fire events when players join, leave, or update their data. Same observer pattern as before

Lobby is "the menu before the game." NGO doesn't really care about that part. Steam handles it.

Steamworks → NGO Handoff

The flow when the lobby owner is ready to start:

1. Lobby owner calls `NetworkManager.StartHost()` — they become both host and server
2. Steam handles the actual **socket-level connection** to each lobby member
3. Other lobby members call `NetworkManager.StartClient()` and connect to the hosts
SteamId
4. From here on, NGO takes over. Spawning objects, syncing variables, RPCs, scene loads, etc.

The clean split: **Steam owns the connection. NGO owns the gameplay.**

NetworkManager.SceneManager

Switching scenes in multiplayer is its own problem. NGO has a SceneManager for it.

- The **server** calls a load on `NetworkManager.Singleton.SceneManager`
- Every connected client transitions to the new scene automatically
- I don't manually orchestrate "now load this scene" per client

This is how I move everyone from the lobby scene into the actual match scene at the same time.

Waiting for Everyone — OnLoadEventCompleted

A real bug I hit: I called `LoadScene` on the server and immediately started the match.

- Most clients loaded fast
- One client took **15+ seconds** to finish loading the new scene
- That player joined the match already in progress, missing the start

The fix: subscribe to `OnLoadEventCompleted`. It fires once **every connected client has finished loading** the scene. I wait for that callback before unfreezing the match.

The observer pattern, once again, saves the day.

NetworkTransform

NetworkTransform is the easy-mode position sync.

- Drop the component on a networked object
- It sends transform updates **every physics tick** from the owner to everyone else
- Other clients **interpolate** between updates so motion looks smooth

Fine for player characters and most physics objects. The "I just want this thing's position to match across machines" tool.

When `NetworkTransform` Falls Apart

I learned this the hard way with spell projectiles.

- On a bad connection, position updates arrive at **inconsistent** times
- The interpolation has gaps and weird timing
- Result: projectiles **stutter, lag, and rubberband** on the receiving client
- Hard to dodge a fireball that teleports

`NetworkTransform` is great for things that turn and accelerate. It's bad for fast, predictable motion.

Skip It For Deterministic Motion

A spell projectile in my game has a **fixed start, direction, and speed**. Given those three, every machine can compute where the projectile is *without* being told every tick.

So instead of `NetworkTransform` :

- Server spawns the projectile and sends each client the **initial conditions once** (start, direction, spawn time)
- Each client **simulates the flight locally** from that point forward
- No per-tick sync, no jitter, no rubberbanding
- The server still owns **collision and damage**. visuals are just visuals

If the motion is deterministic, sync the start, not the result.

What I Learned

- **Spawning** makes a copy of the object on every client, and the same scripts run separately on each one
- Server and clients run the same code: `IsServer` and `IsOwner` are how you gate which machine actually does what
- **NetworkVariables** + listeners + **NGO callbacks** + Steamworks events are all the same observer pattern
- **Steamworks** gave me a free transport and free lobbies: Steam owns the connection, NGO owns the game
- `NetworkManager.SceneManager` + `OnLoadEventCompleted` is how you start the match only after everyone's actually loaded
- `NetworkTransform` is the easy answer, but for deterministic motion you should sync initial conditions and simulate locally

Thanks

Questions?